Book Template

v2.0.0 2026-03-13 MIT

Write beautiful scientific book or thesis with Typst

Mathieu Aucejo

This Typst package is a proposed template for writing thesis dissertations, French habilitations, or scientific books.

Table of Contents

Usage **2**
- I.1 Using `bookly` 2
- I.2 Initializing the template 2
  - I.2.1 Initialization example 5
  - I.2.2 Themes gallery 6
  - I.2.3 Layout 10

Book content **12**
- II.1 Environments 12
- II.2 Parts and chapters 13
- II.3 Tables of contents 15

Helper functions **16**
- III.1 Figure captions 16
- III.2 Subfigures 16
- III.3 Equations 17
- III.4 Information boxes 17
- III.5 Title pages 18
- III.6 Back cover 21
- III.7 Tufte layout 23

Theming **26**
- IV.1 Custom theme definition 26
- IV.2 Template states 27

Dependencies **30**

Part I
Usage

I.1 Using `bookly`

To use the bookly template, you need to include the following line at the beginning of your `typ` file:

```
#import "@preview/bookly:2.0.0": *
```

In `bookly`, the supplement of the `ref` function is set to "none" by default to avoid unexpected behavior when referencing more than one item. However, you can revert this setting by using the following syntax after the template definition:

```
#set ref(supplement: auto)
```

I.2 Initializing the template

After importing bookly, you have to initialize the template by a show rule with the `#bookly` command. This function takes an optional argument to specify the title of the document.

```
#show: bookly.with(
  ...
)
```

```
#bookly(
  ⟨title⟩: "Title",
  ⟨author⟩: "Author Name",
  ⟨theme⟩: "fancy",
  ⟨tufte⟩: false,
  ⟨lang⟩: "fr",
  ⟨fonts⟩: "default-fonts",
  ⟨colors⟩: "default-colors",
  ⟨title-page⟩: none,
  ⟨config-options⟩: "default-config-options"
)[body]
```

Argument

`⟨title⟩: "Title"` `str`

Title of the book or the thesis.

Argument

```
⟨author⟩: "Author Name"                                str
```

Author of the book.

Argument

`(theme): "fancy"` `function`

Theme of the document. Possible values are:
- `fancy` (default)
- `modern`
- `classic`
- `orly` (O'Reilly inspired)
- `pretty`

Argument

`(tufte): false` `bool`

If `true`, the layout of the document is inspired by the works of Edward Tufte (wide margins, sidenotes, etc.).

Argument

`(lang): "en"` `str`

Language of the document.

Supported languages:
- English – `"en"` (default)
- Chinese – `"zh"`
- French – `"fr"`
- German – `"de"`
- Italian – `"it"`
- Portuguese – `"pt"`
- Spanish – `"es"`

Argument

`(fonts): "default-fonts"` `dictionary`

Fonts used in the document. It contains the following keys:
- `body` `str` – Font used for the body text (default: `"New Computer Modern"`)
- `math` `str` – Font used for mathematical equations (default: `"New Computer Modern Math"`)
- `raw` `str` – Font used for raw text (default: `"DejaVu Sans Mono"`)

Argument

`(colors): "default-colors"` `dictionary`

Colors used in the document. It contains the following keys:
- `primary` `color` – Primary color (default: `rgb("#c1002a")`)
- `secondary` `color` – Secondary color (default: `rgb("#dddddd").darken(15%)`)
- `boxeq` `color` – Color of equation boxes (default: `rgb("#dddddd")`)
- `header` `color` – Color used for adapting the color of the document headers (default: `black`)

Argument

⟨title-page⟩: none `content`

Content of the title page.

Argument

⟨config-options⟩: "default-config-options" `dictionary`

Configuration options of the document. It allows a more fine-grained control of some aspects of the template. It contains the following keys:

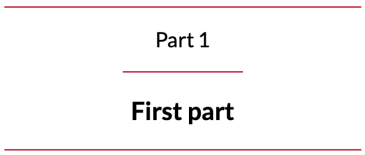
- `part-numbering` `str` – Numbering pattern (default: "1")
- `open-right` `bool` – If `true`, parts start on a right-hand page (default: `true`)

I.2.1 Initialization example

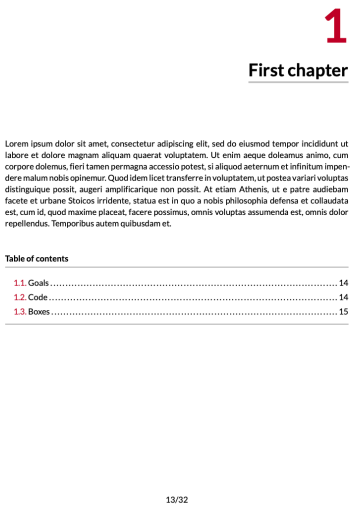
```
#show: bookly.with(
  author: "Author Name",
  fonts: (
    body: "Lato",
    math: "Lete Sans Math"
  ),
  theme: modern,
  lang: "en",
  logo: image("path_to_image/image.png")
)
```

I.2.2 Themes gallery

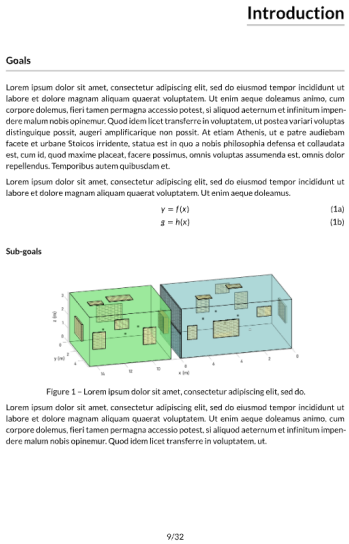
Fancy



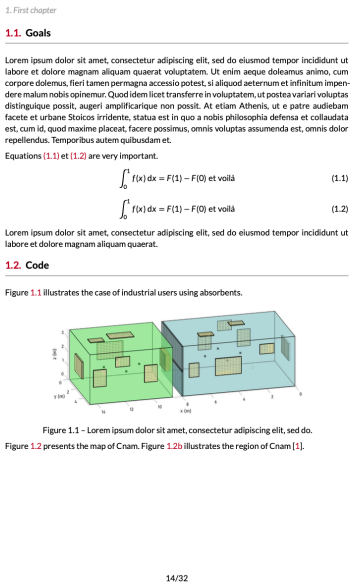
(a) Part



(b) Chapter



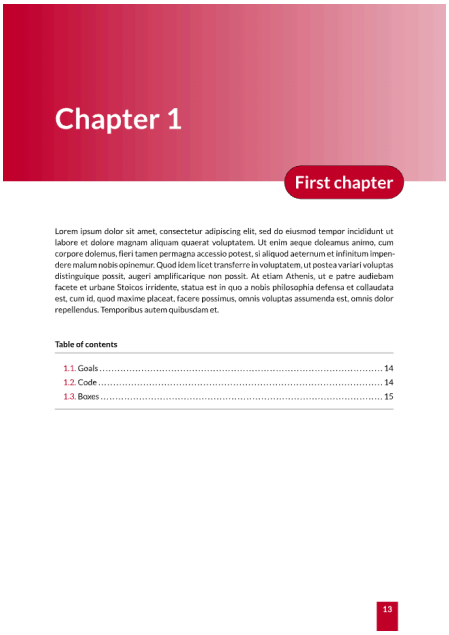
(c) Unnumbered chapter



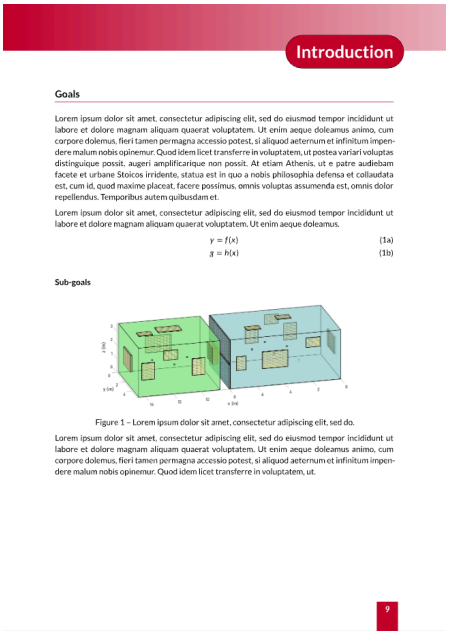
(d) Section

Modern

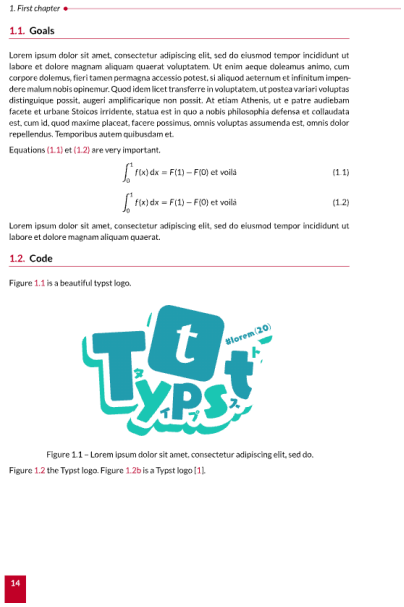
(a) Part



(b) Chapter

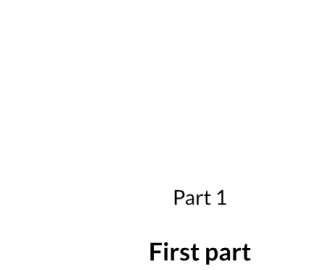


(c) Unnumbered chapter

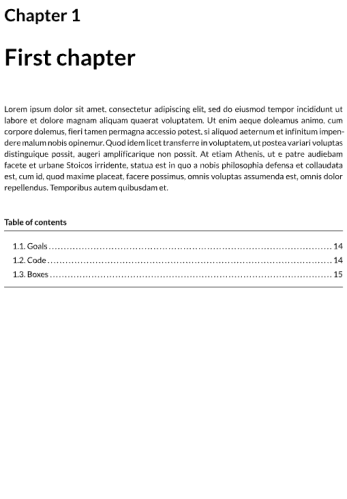


(d) Section

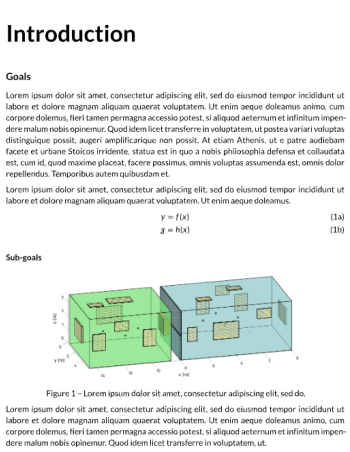
Classic



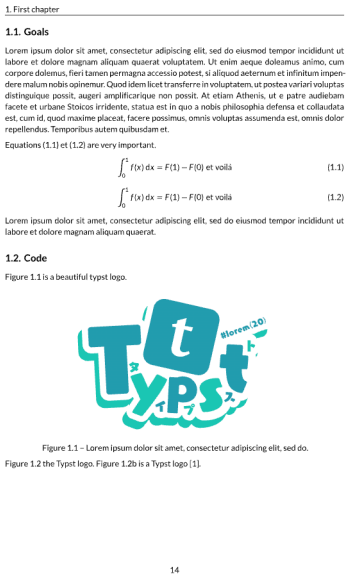
(a) Part



(b) Chapter



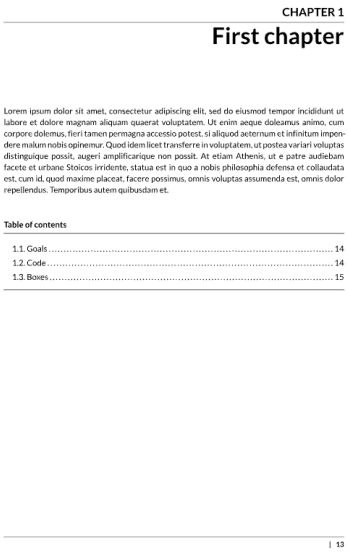
(c) Unnumbered chapter



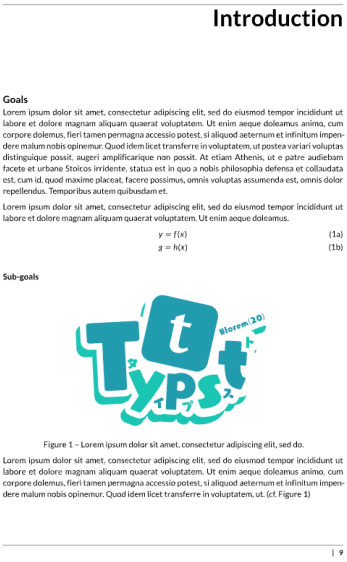
(d) Section

Orly

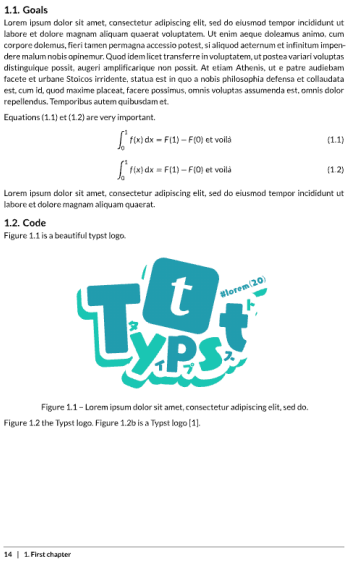
(a) Part



(b) Chapter



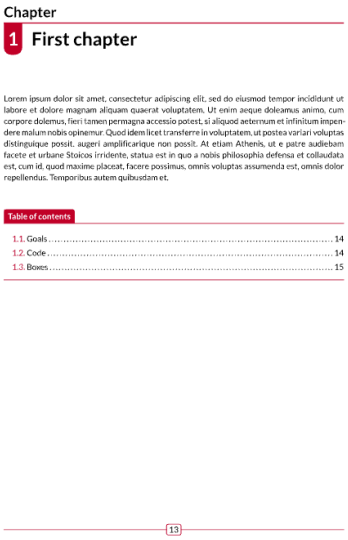
(c) Unnumbered chapter



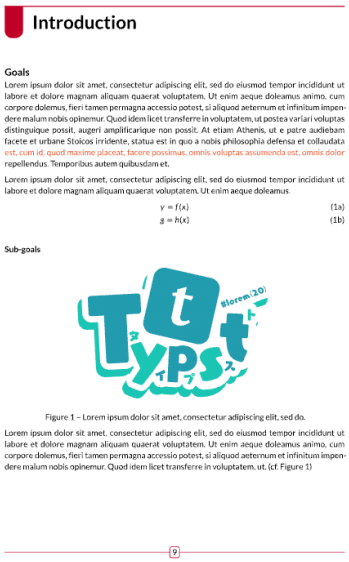
(d) Section

Pretty

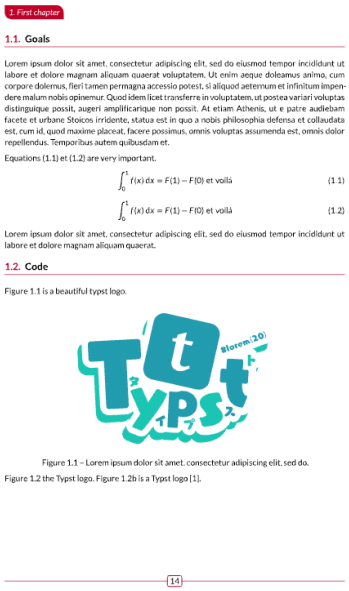
(a) Part



(b) Chapter



(c) Unnumbered chapter



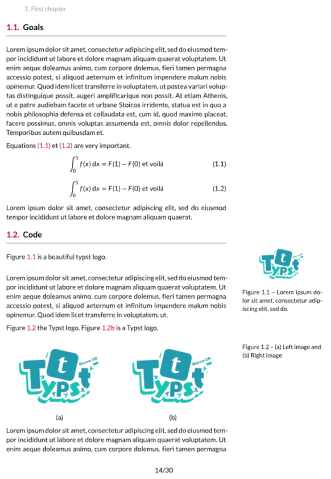
(d) Section

I.2.3 Layout

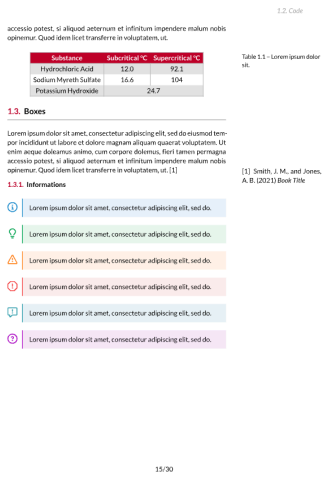
The template currently supports two layouts: `standard` and `tufte`.

The `standard` layout is the default layout, with symmetric margins. It is the most common layout for books and theses. Some examples of the standard layout are presented in Section I.2.2 “Themes gallery”.

The `tufte` layout is inspired by the works of Edward Tufte, which emphasizes simplicity and clarity, often using wide margins for notes and figures. It is particularly suitable for books or theses that require extensive annotations or side comments. To implement the `tufte` layout, the template comes with several helper functions, implementing side notes, side figures, full width blocks, etc. (see Section III.7 for details). Some examples of the `tufte` layout are presented below.



(a) Figures and side figures



(b) Citations

(c) Full width elements

Part II
Book content

The content of the book should be written in the main `typ` file or in additional files. The template provides a basic structure for writing a book.

In general, the section of the main file corresponding to the book content is structured as follows:

```
#show: front-matter

#include "front-content.typ"

#show: main-matter

#tableofcontents

#listoffigures

#listoftables

#part("Main body")

#include "chapter.typ"

#bibliography("bibliography.bib")

#show: appendix

#part("Document appendices")

#include "appendix.typ"
```

The content of the thesis is divided into three main sections: `front-matter`, `main-matter`, and `appendix`. These elements are accompanied by additional functions to facilitate writing.

II.1 Environments

The template provides three environments to structure the thesis content:

1. **front-matter**: environment for preliminary content (cover page, abstract, acknowledgments, etc.). Pages are numbered with Roman numerals and chapters are not numbered. To activate this environment, insert the following command in the main `typ` file at the desired location:

```
#show: front-matter
```

2. **main-matter**: environment for the main content (introduction, tables of contents, chapters, conclusion, bibliography, etc.). Pages and chapters are numbered with Arabic numerals. To activate this environment, insert the following command in the main `typ` file at the desired location:

```
#show: main-matter
```

3. **appendix**: environment for the appendices. Pages are numbered with Roman numerals and chapters are numbered with letters. To activate this environment, insert the following command in the main `typ` file at the desired location:

```
#show: appendix
```

II.2 Parts and chapters

To structure the book content, you can define parts using the `#part` function. To insert a new part, use the following command:

```
#part("Part title")
```

Chapters can also be defined using the standard Typst markup language. This template defines a function `#chapter` that helps you to avoid boilerplate code, such as the manual inclusion of standard elements like title, abstract, and minitoc.

```
#chapter(
  ⟨title⟩,
  ⟨abstract⟩: none,
  ⟨toc⟩: true,
  ⟨numbered⟩: true,
  ⟨label⟩: none
)[body]
```

Argument

`⟨title⟩` `str`

Chapter title.

Argument

`⟨abstract⟩: none` `content`

Summary displayed below the chapter title.

Argument

`(toc): true` `bool`

Indicates whether a mini table of contents should be displayed at the beginning of the chapter.

Argument

`(numbered): true` `bool`

Indicates whether the chapter should be numbered.

Argument

`(label): none` `label`

Label for the chapter.

```
#chapter(
  "First chapter",
  abstract: lorem(20),
  label: <ch:1>
)[
  // Content of the chapter
]
```

If you use a *.typ file for each chapter, you can type at the top of the file the following code.

```
#show: chapter.with("First chapter", abstract: lorem(20), toc: true, label:
<ch:1>)

// Content of the chapter
== First section
```

For unnumbered chapters, you can simply use the `#chapter-nonum` function. This function assumes that you have a *.typ file per chapter.

```
#show: chapter-nonum

// Content of the chapter
= Chapter title
```

`bookly` also provides the `label` `<nonum-sec>` to create unnumbered sections. To use it, simply add the label `<nonum-sec>` after the title of the considered section.

```
== Section title <nonum-sec>
```

The `<nonum-sec>` label only works for sections and not for chapters. When applied to chapters, it breakes the global numbering of the document. For unnumbered chapters, use the `#chapter-nonum` function. instead

II.3 Tables of contents

The template defines several commands to facilitate the creation of tables of contents:

- `#tableofcontents` : Table of contents
- `#listoffigures` : List of figures
- `#listoftables` : List of tables

A mini table of contents is automatically generated by using the command `#minitoc` in a chapter. This function is a wraper of the `#suboutline` function provided by the `suboutline` package.

Part III
Helper functions

III.1 Figure captions

The package include the command `#ls-caption` to manage long and short captions for figures and tables. Short caption are displayed in the list of figures or tables, while long captions are used in the main text and in the table of contents.

```
#figure(
  rect(),
  caption: ls-caption("Long caption", "Short caption")
)
```

> The code of the command `#ls-caption` comes from the Typst book[1] by Sitandr.

III.2 Subfigures

In general, figures are inserted into the document using the `#figure` function from Typst. However, Typst currently does not provide mechanisms for handling subfigures (numbering and referencing). To address this limitation, the template includes a `#subfigure` function that manages subfigures appropriately. This function wraps the `#subpar.grid` function from the `subpar` package.

```
#subfigure(
  figure(image("image1.png"), caption: []),
  figure(image("image2.png"), caption: []), <b>,
  columns: (1fr, 1fr),
  caption: [Figure title],
  label: <fig:subfig>,
)
```

> The example above shows a figure composed of two subfigures. The first subfigure has a caption, while the second has a `label` but no title. The second subfigure can be referenced in the text using the command `@b`.

[1] https://sitandr.github.io/typst-examples-book/book/snippets/chapters/outlines.html?highlight=long#long-and-short-captions-for-the-outline

III.3 Equations

To highlight an important equation, use the #boxeq function.

```
$
  #boxeq[$p(A|B) prop p(B|A) space p(A)$]
$
```

To create an equation without numbering, use the #nonumeq function.

```
#nonumeq[$integral_0^1 f(x) dif x = F(1) - F(0)$]
```

bookly also provides the `label` <nonum-eq> to create unnumbered equations. To use it, simply add the label <nonum-eq> after the equation.

```
$
  integral_0^1 f(x) dif x = F(1) - F(0)
$ <nonum-eq>
```

> The command #nonumeq will be deprecated in a future version in favor of the label <nonum-eq>.

III.4 Information boxes

The template provides several types of boxes to highlight different kinds of content:

- #info-box for remarks;
- #tip-box for tips;
- #warning-box for warnings;
- #important-box for important information;
- #proof-box for proofs;
- #question-box for questions.

```
#info-box[#lorem(10)]
#tip-box[#lorem(10)]
#warning-box[#lorem(10)]
#important-box[#lorem(10)]
#proof-box[#lorem(10)]
#question-box[#lorem(10)]
```

> The appearance of the boxes depends on the selected theme (see the “Themes gallery” section).

The information boxes described above are built using the `#custom-box` function, which allows you to create custom boxes. This generic function takes the following parameters:

```
#custom-box(⟨title⟩: none, ⟨icon⟩: "info", ⟨color⟩: rgb("#1d90d0"))[body]
```

Argument
`⟨title⟩: none` `str`

Name of the box.

Argument
`⟨icon⟩: "info"` `str`

Name of the icon to display in the box.

Available icons are:

- : `"alert"`
- : `"info"`
- : `"question"`
- : `"report"`
- : `"stop"`
- : `"tip"`

Argument
`⟨color⟩: rgb("#1d90d0")` `color`

Box color.

III.5 Title pages

The template provides two functions to create title pages: one for a book and one for a thesis :

```
#book-title-page(
  ⟨subtitle⟩: "Book subtitle",
  ⟨edition⟩: "First edition",
  ⟨institution⟩: "Institution",
  ⟨series⟩: "Discipline",
  ⟨year⟩: "2024",
  ⟨cover⟩: none,
  ⟨logo⟩: none
)[body]
```

Argument
`⟨subtitle⟩: "Book subtitle"` `str`

Subtitle of the book.

Argument
`⟨edition⟩: "First edition"` `str`

Edition of the book.

Argument

⟨institution⟩: "Institution" str

Name of the institution.

Argument

⟨series⟩: "Discipline" str

Name of the series.

Argument

⟨year⟩: "2024" str

Year of publication.

Argument

⟨cover⟩: none image

Cover image of the book.

Argument

⟨logo⟩: none image

Logo of the book.

```
#show: book.with(
  title-page: book-title-page(
    logo: image("path_to_logo/logo.png"),
    cover: image("path_to_image/book-cover.jpg")
  )
)
```

```
#thesis-title-page(
  ⟨type⟩: "phd",
  ⟨school⟩: "School name",
  ⟨doctoral-school⟩: "Name of the doctoral school",
  ⟨supervisor⟩: ("Supervisor name",),
  ⟨cosupervisor⟩: none,
  ⟨laboratory⟩: "Laboratory name",
  ⟨defense-date⟩: "01 January 1970",
  ⟨discipline⟩: "Discipline",
  ⟨specialty⟩: "Speciality",
  ⟨committee⟩: (:),
  ⟨logo⟩: none
)[body]
```

Argument

`⟨type⟩: "phd"` `str`

Type of thesis. Two values are possible:
- `"phd"` for a doctoral thesis
- `"hablitation"` for a French habilitation

Argument

`⟨school⟩: "School name"` `str`

Name of the institution where the thesis was prepared.

Argument

`⟨doctoral-school⟩: "Name of the doctoral school"` `str`

Name of the doctoral school.

Argument

`⟨supervisor⟩: ("Supervisor name",)` `array`

Name of the thesis supervisor(s) or the guarantor of the habilitation.

Argument

`⟨cosupervisor⟩: none` `array`

Name of the thesis co-supervisor(s).

Argument

`⟨laboratory⟩: "Laboratory name"` `str`

Name of the research laboratory.

Argument

`⟨defense-date⟩: "01 January 1970"` `str`

Date of the thesis defense.

Argument

`⟨discipline⟩: "Discipline"` `str`

Name of the discipline.

Argument

`⟨specialty⟩: "Speciality"` `str`

Name of the specialty.

Argument

`⟨committee⟩: (:)` `array`

Name of the thesis committee members. Each element of the array is a `dictionary` with the following keys:

- `name`: Name of the committee member.
- `position`: Position of the committee member (e.g., “Associate Professor”, “Professor”, etc.).
- `affiliation`: Affiliation of the committee member (e.g., “University Name”).
- `role`: Role of the committee member (e.g., “Chair”, “Member”, “Reviewer”).

Argument

`⟨logo⟩: none` `image`

Logo of the institution.

```
#let committee = (
  (
    name: "Hari Seldon",
    position: "Full Professor",
    affiliation: "Streeling university",
    role: "President",
  ),
  (
    name: "Gal Dornick",
    position: "Associate Professor",
    affiliation: "Synnax University",
    role: "Reviewer"
  ),
)

#show: book.with(
  title-page: thesis-title-page(
    supervisor: ("Supervisor A", "Supervisor B"),
    cosupervisor: ("Co-supervisor A", "Co-supervisor B"),
    committee: committee
  )
)
```

For both title pages, the title of the document and its author are automatically generated based on the information given when initializing the template.

III.6 Back cover

A back cover of the document is automatically generated using the `#back-cover` function, which displays information about the thesis (title and author), as well as a summary.

```
#back-cover(⟨resume⟩: none, ⟨abstract⟩: none, ⟨abstracts⟩: (), ⟨logo⟩: none)
```

Argument

⟨abstracts⟩ `dictionary`

Title and summary of the document.

```
#let abstracts-en-fr-de = (
  (
    title: [#set text(lang: "en", region: "gb"); Abstract:],
    text: [#set text(lang: "en", region: "gb")
      This paper presents the objectives, methodology, and main
      results of the work.
    ]
  ),
  (
    title: [#set text(lang: "fr"); Résumé :],
    text: [#set text(lang: "fr")
      Cet article présente les objectifs, la méthodologie et les
      principaux résultats du travail.
    ]
  ),
  (
    title: [#set text(lang: "de"); Zusammenfassung],
    text: [#set text(lang: "de")
      Diese Arbeit beschreibt die Ziele, die Methodik und die
      wichtigsten Ergebnisse.
    ]
  )
)

#back-cover(abstracts: abstracts-en-fr-de, logo: box[logo])
```

Argument

⟨logo⟩ `array`

Logo of the back cover.

```
#let logos = (
  align(left)[#image("images/devise_cnam.svg", width: 45%)],
  align(right)[#image("images/logo_cnam.png", width: 50%)]
)

#back-cover(resume: lorem(10), abstract: lorem(10), logo: logos)
```

III.7 Tufte layout

When the `tufte` layout is selected, several customizations are applied to adapt the appearance of various elements (figures, tables, equations, etc.) to the Tufte style.

`#sidenote(⟨dy⟩: -1.5em, ⟨numbered⟩: true, ⟨label⟩: none)[body]`

Argument
`⟨dy⟩: -1.5em` `length`

Vertical adjustment of the sidenote position.

Argument
`⟨numbered⟩: true` `bool`

Indicates whether the sidenote should be numbered.

Argument
`⟨label⟩: none` `label`

Label of the sidenote.

> When the `tufte` argument is set to `false`, the `#sidenote` function behaves like a standard `#footnote`.

`#sidecite(⟨key⟩, ⟨dy⟩: -1.5em, ⟨supplement⟩: none)`

Argument
`⟨key⟩` `label`

Key of the reference to cite.

Argument
`⟨dy⟩: -1.5em` `length`

Vertical adjustment of the sidecite position.

Argument
`⟨supplement⟩: none` `str`

Supplementary text to add before the citation (e.g., “see”, “e.g.”, etc.).

> When `tufte` argument is set to `false`, the `#sidecite` function behaves like a standard `#cite`.

```
#sidefigure(⟨content⟩, ⟨dy⟩: -1.5em, ⟨label⟩: none, ⟨caption⟩: none)
```

Argument

⟨content⟩ `content`

Content of the figure.

Argument

⟨dy⟩: -1.5em `length`

Vertical adjustment of the sidefigure position.

Argument

⟨caption⟩: none `content`

Caption of the figure.

Argument

⟨label⟩: none `label`

Label of the figure.

```
#fullfigure(⟨content⟩, ⟨label⟩: none, ⟨caption⟩: none)
```

Argument

⟨content⟩ `content`

Content of the figure.

Argument

⟨caption⟩: none `content`

Caption of the figure.

Argument

⟨label⟩: none `label`

Label of the figure.

> When `tufte` argument is set to `false`, `#sidefigure` and `#fullfigure` behave like a standard `#figure`.

Part IV
Theming

The theming system is designed to be flexible and customizable, allowing users to define their own themes.

IV.1 Custom theme definition

To implement a custom theme, you have to define a function that includes the `show` and `set` rules defining the style of the document (headings, footnotes, references, ...). Basically, a theme should be structured as follows:

```
// my-theme.typ
#import "@preview/bookly:2.0.0": *

#let my-theme(colors: default-colors, it) = {
  // Update the theme state
  states.theme.update("custom")

  // Heading Level 1 style
  show heading.where(level: 1): it => {
    // Page break before each part
    if not states.open-right.get() {pagebreak(weak: true)}
    ...
  }

  // Heading Level 2 style
  show heading.where(level: 2): it => {...}

  // Heading Level 3 style
  show heading.where(level: 3): it => {...

  // Outline entry style
  show outline.entry: it => {...}

  // Other show and set rules
  ...

  it
}
```

You can also define your own functions such as `#part`, `#minitoc` and other elements of the document.

> Examples of theming are available in the Github repository[2] of the template.

Then, you can initialize the template with your custom theme as follows:

```
#import "path_to_file/my-theme.typ": *

#show: bookly.with(
  theme: my-theme,
  ...
)
```

> If you use a multiple files structure with a *.typ file for each chapter, you can type at the top of each file the following code to access the functions like `#part` or `#minitoc` defined in the theme file.
>
> ```
> #import "path_to_file/my-theme.typ": *
> ```

IV.2 Template states

`bookly` provides some states that can be useful when designing a custom theme. The states are used to store information about the current state of the document. They are collected in a `dictionary`. The following states are available:

- `states.author` – `str`: Author of the document.
- `states.colors` – `dictionary`: Color scheme for the document.
- `states.counter-part` – `str`: Counter for parts.
- `states.localization` – `dictionary`: Dictionary of terms used in the document (e.g., "chapter", etc.) in the selected language.
- `states.in-outline` – `bool`: Indicates whether the current section is in the outline.
- `states.isappendix` – `bool`: Indicates whether the current section is an appendix.
- `states.isfrontmatter` – `bool`: Indicates whether the current section is front matter.

> If you need to use a language that is not supported by default, you can modify the `states.localization` dictionary when initializing the template.

[2] https://github.com/maucejo/bookly

For example, to add support for Dutch, you can do the following `#states.localization.update(json("path_to_file/dutch.json"))`. The JSON file should contain the translations of the terms used in the document. For the english version, the JSON file is as follows:

```
{
    "and": " and ",
    "appendix": "Appendix",
    "authored": "authored by",
    "chapter": "Chapter",
    "committee": "Defense committee",
    "cosupervisor": "Co-supervisor:",
    "cosupervisors": "Co-supervisors:",
    "defended": "defended on",
    "discipline": "Discipline:",
    "doctoral-school": "DOCTORAL SCHOOL",
    "habilitation": "French Habilitation to supervise research",
    "lof": "List of figures",
    "lot": "List of tables",
    "note": "Note",
    "part": "Part",
    "phd": "Doctoral thesis",
    "proof": "Proof",
    "specialty": "Specialty:",
    "sponsor": "Sponsor:",
    "sponsors": "Sponsors:",
    "supervisor": "Supervisor:",
    "supervisors": "Supervisors:",
    "tip": "Tip",
    "toc": "Table of contents",
    "version-usage": "This version of  can be viewed and downloaded free of charge for personal use only. It must not be redistributed, sold, or used in derivative works.",
    "warning": "Warning"
}
```

- `states.num-heading` – `str` : Numbering pattern for headings.
- `states.num-pattern` – `str` : Numbering pattern for sections.
- `states.num-pattern-eq` – `str` : Numbering pattern for equations.
- `states.num-pattern-fig` – `str` : Numbering pattern for figures.
- `states.num-pattern-subfig` – `str` : Numbering pattern for subfigures.
- `states.open-right` – `bool` : Indicates whether parts and chapters start on a right-hand page.
- `states.page-numbering` – `str` : Numbering pattern for pages.
- `states.part-numbering` – `str` : Numbering pattern for parts.

- `states.sidenotecounter` – `int`: Counter for sidenotes.
- `states.title` – `str`: Title of the document.
- `states.theme` – `str`: Current theme of the document.
- `states.tufte` – `bool`: Indicates whether the current layout is Tufte style.

> `bookly` also comes with a function `#reset-counters` to reset the counters for equations, figures, tables, sidenotes, and footnotes.

Part V

Dependencies

The `bookly` template relies on several Typst packages to provide additional functionalities:

- `drafting:0.2.2`: for tufte layout.
- `hydra:0.6.2` : for bibliography management.
- `equate:0.3.2` : for advanced equation numbering.
- `itemize:0.2.0"`: for lists and enumerations customization.
- `showybox:2.0.4` : for custom boxes.
- `suboutline:0.3.0` : for mini tables of contents in chapters.
- `subpar:0.2.2` : for subfigures.